

CRR Pipeline Layers Explained

Two parallel pipelines -- same conceptual framework, different methods and data

Overview

The Deep Scholar project has two layers that share the same core concept -- $\text{edge} = p - q$ and Kelly position sizing -- but they are independent pipelines that arrive at p using completely different approaches. Layer 2 does NOT build on top of Layer 1; they are parallel implementations of the same economic idea.

Shared Concept in Both Layers:

$\text{edge} = p - q$ | Trade when $\text{edge} > \text{transaction costs}$ | Size with Kelly criterion | Halt on drawdown limit

Layer 1 CRR Theoretical Pricing Pipeline

One AMD contract snapshot -- mathematical / no-arbitrage

- Builds the CRR binomial tree to compute V_{model} (theoretical fair price)
- Computes Greeks: delta, gamma, theta, vega from the tree
- Computes mispricing $\text{edge} = (V_{\text{model}} - V_{\text{market}}) / V_{\text{market}}$
- Sizes the position with full / half / quarter Kelly criterion
- Runs Monte Carlo simulation of P&L over many paths
- Data: 4 AMD contracts from a single moment in time (Fidelity tickets)
- Method: Mathematical -- CRR no-arbitrage formula

Layer 2 ML Empirical Pricing Pipeline

5 years of AAPL historical options data -- machine learning / empirical

- Does NOT use the CRR binomial tree
- Trains XGBoost to estimate $p_{\text{independent}}$ (probability option expires ITM)
- Detects herding regimes from RV-IV spread (`bias_detector.py`)
- Subtracts real transaction costs to get net edge (`micro_cost.py`)
- Calibrates model probabilities with Platt scaling (`calibration.py`)
- Applies regime-conditional trade policy: normal 2%, herding 8% min edge
- Runs walk-forward backtest: 12-month train, 3-month test windows
- Data: 47,578 AAPL rows (2016-2020) -- historical outcomes
- Method: Machine learning -- XGBoost + Platt scaling calibration

Layer 1: p from the CRR Formula

The CRR risk-neutral probability p^* is derived mathematically from the no-arbitrage condition. It is not a forecast -- it is the unique probability that makes the discounted option price a martingale:

```
u = exp(sigma * sqrt(dt))      # up factor
d = 1 / u                      # down factor
p* = (exp(r*dt) - d) / (u - d) # risk-neutral probability
V_model = backward_induction(p*, u, d, K, S0, N)
edge = (V_model - V_market) / V_market
```

Layer 2: p from XGBoost Trained on Historical Outcomes

$p_{\text{independent}}$ is an empirical estimate of the physical probability the option expires in the money, trained on 5 years of actual AAPL expiration outcomes. It is NOT derived from the no-arbitrage formula:

```
features = [moneyness, log_dte, IV, RV30,
            rv_iv_spread, half_spread, log_volume]
label     = 1 if option expired ITM else 0    # historical outcome

model = XGBoost + CalibratedClassifierCV(cv=3, method='sigmoid')
model.fit(X_train, y_train)    # 38,062 training rows

p_independent = model.predict_proba(X_test)[: , 1]
q_market      = option_mid / strike    # market-implied probability proxy
raw_edge      = p_independent - q_market
net_edge      = raw_edge - transaction_costs
```

Side-by-Side Comparison

	Layer 1	Layer 2
p comes from	CRR no-arbitrage formula	XGBoost trained on 5 yrs of AAPL data
q comes from	AMD market bid-ask mid	AAPL bid-ask mid at each historical date
Data	4 AMD contracts (one moment in time)	47,578 AAPL rows (2016-2020)
Method	Mathematical / no-arbitrage	Machine learning / empirical
Edge signal	$(V_{\text{model}} - V_{\text{market}}) / V_{\text{market}}$	$p_{\text{independent}} - q_{\text{market}}$ after costs
Sizing	Full / half / quarter Kelly	Quarter Kelly (backtest conservatism)
Risk controls	VaR limit, drawdown halt	VaR 5%, drawdown halt 15%
Output	Greeks, edge, kelly.csv, simulation.csv	call/put_model.pkl, pnl_curve.png, trades.csv

Key Insight: Same Question, Different Answers

Both layers ask: 'Is the market pricing this option correctly?' Layer 1 answers using the theoretical no-arbitrage price -- if the market deviates from what the CRR formula says it should cost, there is an edge. Layer 2 answers using historical evidence -- if the ML model's predicted ITM probability differs from what the market implies, there is an edge. They are two independent methods of finding the same thing.

Module	What it does	Key output
data.py	Loads AMD contract params from config.yaml	S, K, r, sigma, T, N
tree.py	Builds CRR binomial lattice, backward induction	V_model (fair price)
greeks.py	Numerical Greeks from tree perturbation	delta, gamma, theta, vega
edge.py	Mispricing: $(V_model - V_market) / V_market$	edges.csv
kelly.py	$f^* = (p*b - q) / b$ position sizing	kelly.csv
simulation.py	Monte Carlo P&L simulation over many paths	simulation.csv, plots
-- Layer 2 modules below --		
market_data.py	Load AAPL CSV, compute RV30, label ITM/OTM	Prepared DataFrame
p_estimator.py	Train XGBoost + Platt scaling on ITM outcomes	call/put_model.pkl
bias_detector.py	RV-IV spread, iv_mom, regime classification	regime_analysis.png
micro_cost.py	Round-trip cost: spread + fee + slippage	net_edge per trade
calibration.py	Brier score, ECE, reliability diagram	calibration_curve.png
policy.py	Regime-gated TRADE / NO TRADE decision	Signal + Kelly sizing
backtest.py	Walk-forward 12-mo train / 3-mo test windows	pnl_curve.png, trades.csv

Potential Integration Point (Not Yet Implemented)

The two layers are currently independent. One natural extension would be to use the CRR model price as an additional feature inside the Layer 2 XGBoost model -- feeding the theoretical edge $(V_model - V_market) / V_market$ as a feature column alongside moneyness, IV, and RV. This would let the ML model learn when the no-arbitrage signal is more or less reliable depending on market regime. As built today, Layer 1 runs on AMD data and Layer 2 runs on AAPL data; integrating them would require a common dataset with both theoretical and empirical signals computed for the same contracts.